

Aakar — Roadmap (v1)

What's planned next, in roughly the order it will land. Phases are calendar guidance, not contracts; items can move when reality intervenes. Every item below is anchored to a concrete Sadhaka pain or platform need we have already seen in v1. Mythic primary, English in parens on first occurrence per section.

Phase 1 — within 2 weeks

Goals: lower the friction for the first Sadhakas of a new Mandala, sharpen the Samvada UX, and finish the long tail of v1 polish.

1.1 Auto-grant session Vidyas on Pravesha (login)

Today, an Acharya (tenant admin) must explicitly grant `cap.web_login` to a Sadhaka before they can run any login-bearing Sutra. In practice, every Sadhaka needs it. Phase 1 will auto-grant the read-only "session" Vidyas (`web_login` , `http.fetch_session` , `screenshot`) on Sadhaka creation, with an opt-out toggle.

1.2 Kosha (vault) redesign for non-login Vidyas

The current Kosha is login-centric. Phase 1 widens the schema so a Mandala can store API keys, OAuth refresh tokens, and per-site cookies — anything a Vidya declares it needs. The UI moves to a generic "Site → Credential type → Entry" tree.

1.3 In-Samvada credential widget

When the Drashtri pauses with a `picker` Aahvaana because the Kosha is missing an entry, the Samvada will render a credential entry form inline rather than redirecting the Sadhaka to `/admin/grants` . The form posts directly to the Kosha and resolves the Aahvaana in one step.

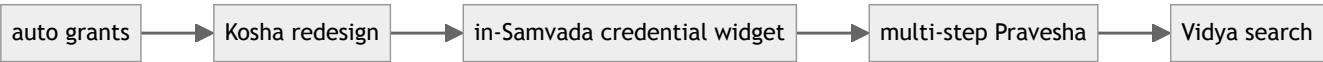
1.4 Multi-step Pravesha (login)

Some sites require username on page A, password on page B, OTP on page C. The current `web_login` Vidya assumes a single form. Phase 1 introduces `LoginFlow` as a chain of Pravesha steps a Vidya can declare.

1.5 Vidya search (Anveshana) in the Pracharya's surface

The Pracharya's catalog page will gain a top-of-page search box backed by the same FAISS / pgvector index the Drashtri uses. This makes it cheap to discover near-duplicate Vidyas before

publishing a new one.



Phase 2 — within 1 month

Goals: production hardening. Replace dev shortcuts with services that scale, add the observability story, and unlock more expressive Yantras.

2.1 OpenTelemetry observability

Phase 2 introduces full OpenTelemetry support across both backends and the frontend. This is the single biggest visibility upgrade between v1 and the next version.

2.1.1 What gets instrumented

Layer	Spans	Notable attributes
FastAPI	one root span per request	<code>http.method</code> , <code>http.route</code> , <code>tenant.id</code> , <code>user.id</code> , <code>request_id</code>
Drashtri (planner)	spans for "plan" plus child spans per LLM call and tool call	<code>planner.strategy</code> (oneshot or agentic), <code>planner.candidate_count</code>
Validator	one span per Yantra validation	<code>dag.node_count</code> , <code>validator.failed_rule</code>
Karta (executor)	one span per Yajna, child spans per node, grandchild per Aahvaana wait	<code>run.id</code> , <code>node.id</code> , <code>capability.id</code> , <code>signal.kind</code>
Vidya	one span per Vidya invocation	<code>capability.id</code> , <code>vault.read_count</code>
Browser worker	span per Playwright step	<code>browser.action</code> , <code>browser.selector_strategy</code> , <code>browser.duration_ms</code>
HTTP worker	span per outbound request	standard HTTP semantic conventions
Frontend	spans for navigation, query fetches, Smriti SSE lifecycle	<code>page.path</code> , <code>query.key</code>

2.1.2 Metrics

Metric	Type	Dimensions
<code>aakar.runs.started</code>	counter	<code>tenant.id</code> , <code>capability.id</code>
<code>aakar.runs.duration_ms</code>	histogram	<code>tenant.id</code> , <code>capability.id</code> , <code>status</code>
<code>aakar.runs.failed</code>	counter	<code>tenant.id</code> , <code>capability.id</code> , <code>failed_node_id</code>
<code>aakar.signals.published</code>	counter	<code>signal.kind</code>
<code>aakar.signals.wait_ms</code>	histogram	<code>signal.kind</code>
<code>aakar.planner.tokens_in</code>	counter	<code>planner.strategy</code>
<code>aakar.planner.tokens_out</code>	counter	<code>planner.strategy</code>
<code>aakar.browser.pool.in_use</code>	gauge	(none)
<code>aakar.vault.reads</code>	counter	<code>tenant.id</code>

Histograms use the OTel default boundaries; we will tune buckets after a week of real data.

2.1.3 Logs

The structured logger (`structlog`) gets bridged into OTel logs. Every log record inherits the active span context, so a log line in the Karta automatically carries the `run.id` and `node.id` of the node that emitted it. Trace-to-log correlation lets ops jump from a slow span to the exact log lines for that node.

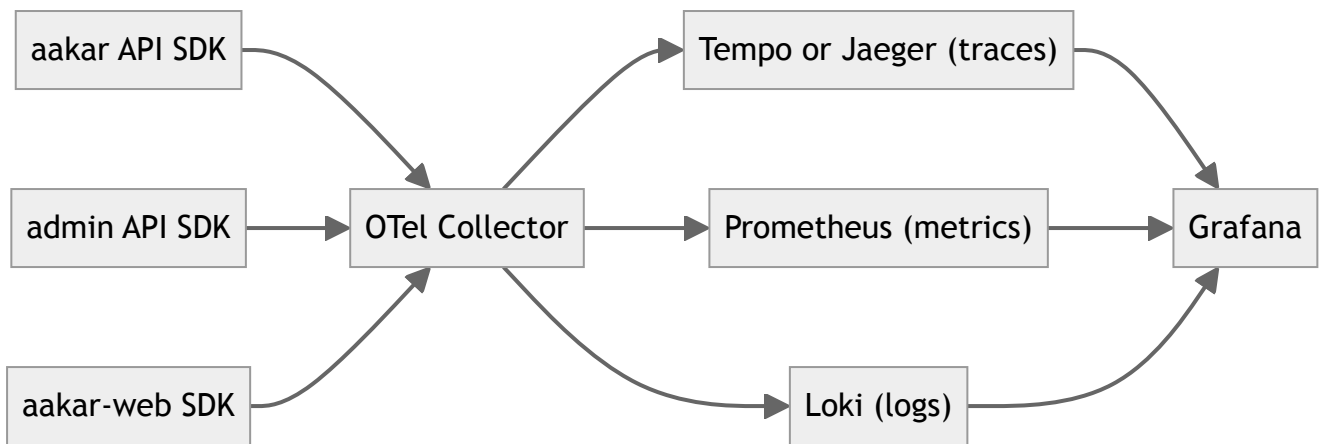
2.1.4 Resource attributes

Every signal (trace, metric, log) carries a stable resource bundle:

- `service.name` — `aakar-api` , `admin-app-api` , `aakar-web` .
- `service.version` — Git short SHA from build.
- `deployment.environment` — `dev` , `staging` , `prod` .
- `service.instance.id` — pod name in cloud, hostname in dev.

`tenant.id` is allowed as a span and metric attribute (low cardinality at our scale). `run.id` is allowed on spans only (high cardinality).

2.1.5 Pipeline and exporters



The collector runs as a sidecar in dev and as a per-node DaemonSet in cloud. Backends are pluggable; the exporters above are the v1 picks but the choice does not leak into application code.

2.1.6 Sampling

Default is head-based at 10% for normal traffic plus always-on for any trace whose root span's `error` attribute is true. Tail-based sampling on the collector retains slow traces (greater than the 95th percentile) and traces from a configurable list of "VIP" Mandalas for debugging.

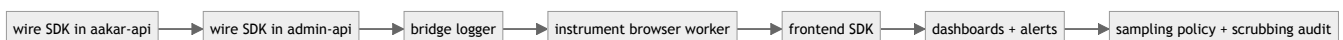
2.1.7 Privacy

Span attributes are never allowed to carry credentials, cookie values, Kosha payloads, or full file contents. A small allowlist plus `process.env.OTEL_SCRUB_KEYS` controls scrubbing at SDK level. PII fields in Smriti payloads are redacted before they ever reach the OTel pipeline.

2.1.8 Local developer experience

`docker compose up otel` brings up the collector, Tempo, Prometheus, Loki, and Grafana with seeded dashboards. Hitting the API in dev produces traces that show up in Grafana within seconds. The full stack is opt-in; running without `OTEL_EXPORTER_OTLP_ENDPOINT` set is a no-op.

2.1.9 Rollout sequence



2.2 Temporal-backed Karta

The Karta Protocol already has a `TemporalKarta` stub. Phase 2 finishes it, with workflow types per Vidya and child workflows for sub-Yantras. Local dev keeps `LocalKarta`.

2.3 Cron Sankalpas

Expose a per-Sutra cron field. The Pracharya's surface gets a "Schedules" page. Cron is backed by the same Temporal cluster that powers the Karta.

2.4 Production Kosha (vault)

Replace the dev plaintext-on-disk adapter with a managed-KMS-backed driver: per-Mandala data key encrypts the secret blob; the data key is wrapped by a master key in a managed KMS; the master key never enters the application process. Cipher rotation is staged so old entries can be re-encrypted without downtime.

2.5 Warm browser pool

Today every Yajna boots a fresh Chromium context (about 1.5 to 3 seconds). Phase 2 adds a pool of pre-warmed contexts isolated by Mandala. Eviction on Mandala-scope conflicts is enforced.

2.6 Branch and for_each in Yantras

The Yantra schema gets two new node types: `branch` (route by predicate) and `for_each` (fan out over a list and join). The validator and Karta learn to handle them; the Drashtri learns to emit them when Sankalpas call for "for each row in the file ...".



Phase 3 — 3 to 6 months

Goals: turn Aakar from a single-deployment product into a platform that can host external publishers and serve regulated Mandalas.

3.1 Vidya marketplace

A signed-publisher model where third-party teams can submit Vidyas. Each submission goes through code review, schema validation, a security scan, and a test in a sandbox Mandala before becoming visible.

3.2 Sakshi (audit) surface

Today the `audit_log` (Sakshi) table is queryable by SQL but not exposed in product. Phase 3 builds an Acharya audit view with filter by Sadhaka, action, and time, plus signed-export for compliance.

3.3 Yajna replay

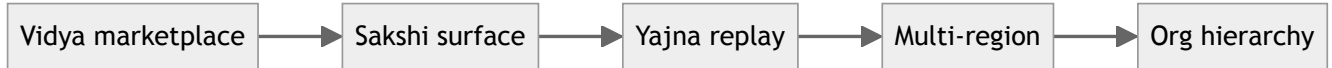
Given a `run_id`, replay produces a deterministic re-Yajna from persisted Yantra and inputs against a recorded mock of the third party. Useful for incident triage and regression tests.

3.4 Multi-region

The product currently assumes a single deployment per environment. Phase 3 introduces region affinity: a Mandala is pinned to a region; cross-region queries are explicit; the Kosha is region-local.

3.5 Org hierarchy

Mandalas can be grouped into orgs. Adhikaras and quotas can be set at the org level and inherited; per-Mandala exceptions still work.



Spike candidates (un-prioritized)

- Streaming LLM responses in Samvada with cancellation and partial Yantra previews.
- Drashtri self-critique pass before emitting a Yantra.
- Vidya "lint" checks (selector freshness, fixture currency).
- A small mobile companion app for captcha resolution and Yajna notifications.
- Feature-flag service plus per-Mandala rollout percentages.
- Out-of-the-box SAML / OIDC.

Engineering policies

- **One PR per logical change.** Bundle only when a split would be churn.
- **Integration tests hit a real database.** No mocks at the persistence boundary.
- **Migrations are reviewed for online-safety.** Adding a NOT NULL column to a hot table is a multi-PR sequence.
- **No silent fallbacks for security.** A missing Vidya or a missing Adhikara returns an explicit error.
- **Comments justify the WHY only.** WHAT is in the code; PR description carries context.
- **No hard-coded credentials anywhere.** Kosha or env, never in code or tests.

Decision log (open questions)

ID	Question	Owner	Target decision date
RFC-04	Are we ok with shipping TemporalKarta before warm-pool isolation is proven?	Platform	2026-05-30
RFC-05	Where does Yajna replay state live: same DB or separate journaling store?	Platform	2026-06-15
RFC-06	Cron Sankalpas in admin surface or in aakar API?	Platform	2026-05-25
RFC-07	OTel span naming: do we adopt OpenTelemetry Semantic Conventions verbatim or define an Aakar overlay?	Observability	2026-05-20

What this roadmap is not

- It does not promise dates. Items move when scope changes or facts on the ground move.
- It does not authorize work outside the listed phases without an explicit RFC.
- It does not replace the HLD or LLD. Items here that touch the architectural spine require a corresponding HLD or LLD update before merge.